

Handout: C-Entwicklungsumgebung unter Windows

Setup: VS Code + WSL (Windows Subsystem for Linux)

Zielsetzung

Erstellung einer professionellen Entwicklungsumgebung, bei der die Benutzeroberfläche unter **Windows** läuft (VS Code), die Kompilierung und Ausführung des Codes jedoch in einer echten **Linux-Umgebung (WSL)** erfolgt. Dies entspricht dem Standard in der professionellen Softwareentwicklung.

Schritt 1: WSL (Windows Subsystem for Linux) installieren

Bevor wir programmieren, benötigen wir ein Linux-System innerhalb von Windows.

1. Öffne die **PowerShell** oder die **Eingabeaufforderung** als Administrator (Rechtsklick auf Start -> Terminal/PowerShell (Administrator)).
2. Gib folgenden Befehl ein:

```
wsl --install
```
3. **Wichtig:** Wenn dies das erste Mal ist, musst du deinen Computer **neu starten**.
4. Nach dem Neustart öffnet sich automatisch ein Linux-Terminal (meist Ubuntu). Erstelle dort einen **Benutzernamen** und ein **Passwort** (das Passwort wird beim Tippen nicht angezeigt!).

Schritt 2: Linux-Werkzeuge installieren (Compiler & Debugger)

Jetzt installieren wir die Werkzeuge, die den C-Code in ein ausführbares Programm verwandeln.

1. Öffne dein **Ubuntu-Terminal** (über das Startmenü suchen).
2. Aktualisiere die Paketlisten:

```
sudo apt update && sudo apt upgrade -y
```
3. Installiere das Paket **build-essential** (enthält den **gcc** Compiler und **make**) sowie den Debugger **gdb**:

```
sudo apt install build-essential gdb -y
```
4. **Prüfung:** Gib `gcc --version` ein. Wenn eine Versionsnummer erscheint, ist alles bereit!

Schritt 3: VS Code konfigurieren

VS Code muss nun lernen, mit Linux zu kommunizieren.

1. Installiere **Visual Studio Code** unter code.visualstudio.com.
2. Öffne VS Code.
3. Gehe links auf das **Extensions-Icon** (das Quadrat-Symbol) oder drücke **Strg+Shift+X**.
4. Suche und installiere folgende Erweiterungen:
 - **WSL** (von Microsoft) – *Essentiell für die Verbindung.*
 - **C/C++** (von Microsoft) – *Für IntelliSense (Code-Vervollständigung) und Debugging.*

Schritt 4: Der Workflow (Das erste Programm)

Wichtig: Arbeite niemals in deinen Windows-Ordern (wie `C:\Users\...`), wenn du in WSL arbeitest. Nutze das Linux-Dateisystem für maximale Geschwindigkeit!

1. Öffne das Ubuntu-Terminal.
2. Erstelle einen Projektordner und wechsle hinein:

```
mkdir mein_projekt && cd mein_projekt
```

3. Öffne diesen Ordner direkt in VS Code:

```
code .
```

(Beim ersten Mal dauert es einen Moment, da VS Code den "VS Code Server" in Linux installiert.)

4. **Achte auf den Statusbalken:** Links unten in VS Code sollte nun in blau **"WSL: Ubuntu"** stehen.

Test-Code erstellen:

1. Erstelle eine neue Datei: `hello.c`.
2. Schreibe folgenden Code hinein:

```
#include <stdio.h>

int main() {
    printf("Hallo aus der WSL-Welt!\n");
    return 0;
}
```

3. Speichere die Datei (**Strg + S**).

Schritt 5: Kompilieren und Ausführen

Wir nutzen das integrierte Terminal in VS Code (Strg + ö oder Terminal -> New Terminal).

1. **Kompilieren:** Gib im VS Code Terminal ein:

```
gcc hello.c -o hello
```

(Dies erstellt eine ausführbare Datei namens *hello*.)

2. **Ausführen:** Gib ein:

```
./hello
```

3. **Ergebnis:** Du solltest Hallo aus der WSL-Welt! sehen.

Cheat Sheet (Befehlsübersicht)

Befehl	Bedeutung
<code>gcc datei.c -o programm</code>	Kompiliert die Datei <code>datei.c</code> zu <code>programm</code>
<code>./programm</code>	Startet das Programm im aktuellen Ordner
<code>ls</code>	Listet alle Dateien im aktuellen Ordner auf
<code>cd ..</code>	Geht einen Ordner nach oben
<code>pwd</code>	Zeigt den aktuellen Pfad an
<code>rm programm</code>	Löscht die ausführbare Datei

Troubleshooting

- **"gcc command not found":** Du hast Schritt 2 vergessen oder `sudo apt update` nicht ausgeführt.
- **VS Code zeigt keine C-Features:** Stelle sicher, dass die "C/C++" Erweiterung **im WSL-Modus** installiert ist (in der Extensions-Liste muss stehen: *"Install in WSL: Ubuntu"*).
- **Berechtigungsprobleme:** Versuche niemals, Dateien unter `/mnt/c/...` zu bearbeiten, wenn du in WSL arbeitest. Nutze immer dein Home-Verzeichnis (`~/...`).